# STREAMBHARAT

OTT Streaming Analytics Platform

# TEAM DISTRIBUTION & IMPLEMENTATION GUIDE

Six members • Questions Q1–Q20 • Project 4 workbook

| | |
|---|---|
| Project Lead | Ishan |
| Technical Specialist | Rutik |
| Content & Discovery | Harshvardhan |
| Subscriptions & Finance | Kshitij Jaisawal |
| CDN & Search Analytics | Siddhi |
| Engagement & Data Quality | Ankita |

**Purpose:** This guide assigns ownership for all 20 workbook questions and gives each member an introduction, implementation sequence, expected output and key concepts.

# 1. Project overview

StreamBharat is an OTT analytics project. The team processes catalogue, playback, subscription, advertising, search and CDN data using PySpark and Delta Lake. The outputs support watch-time analysis, subscriber metrics, content rankings, royalties, recommendations and churn prediction.

## Core flow

| | | | |
|---|---|---|---|
| **Raw data**<br>JSON / CSV / logs | **Bronze**<br>Ingested data | **Silver**<br>Clean data | **Gold**<br>Business metrics |

The workbook setup uses Python 3.10+, Java 17, PySpark 3.5, Delta Lake and Pytest. Q17–Q18 use the supplied simulator with Spark Structured Streaming. This guide follows Workbook 2; it does not add Kafka, Airflow, PostgreSQL or Metabase as required components.

# 2. Question ownership

| Member | Workstream | Questions |
|---|---|---|
| Ishan | Project lead; ingestion, streaming, integration | Q4, Q17, Q20 |
| Rutik | Playback engineering and Spark optimization | Q2, Q5, Q12, Q14 |
| Harshvardhan | Content modelling and discovery | Q1, Q8, Q10, Q16 |
| Kshitij Jaisawal | Subscriptions, finance and live ad pacing | Q6, Q11, Q15, Q18 |
| Siddhi | CDN processing and search analytics | Q3, Q7 |
| Ankita | Engagement metrics, advertising and quality | Q9, Q13, Q19 |

**Q20:** Ishan owns end-to-end pipeline integration and final delivery. Rutik provides technical support for the churn-model implementation.

# 3. Common setup and working rules

• Extract StreamBharat_dataset.zip; place its data folder inside the project directory and read the dataset README.

• Use Python 3.10+ and Java 17. Create a virtual environment and install: pip install "pyspark==3.5.*" delta-spark pytest.

• Create one shared SparkSession helper in src/common/spark_session.py. All modules must use the same Spark configuration.

• Create data/raw, data/stream, lake/bronze, lake/silver, lake/gold, src/ and tests/ directories.

• Keep raw files unchanged. Agree on each output table's name, schema, data types and storage path before parallel development.

• Use small DataFrames to test transformations before processing the full dataset.

• Give each streaming query its own stable checkpoint directory. Do not reuse a checkpoint for another query.

• A question is complete only when its code runs, its expected output exists, the result is validated and the owner can explain the logic.

# 4. Ishan — Project Lead

Assigned: Q4, Q17, Q20

### *Q4. Incremental playback ingestion*

**Introduction:** Process only newly arriving playback files, write them to Delta and resume safely using checkpoints.

• **Step 1:** Read playback files using Spark Structured Streaming with an explicit schema.

• **Step 2:** Write to a Bronze Delta table partitioned by event date and configure a dedicated checkpoint location.

• **Step 3:** Use trigger(availableNow=True) so the job processes available new files and then stops.

• **Step 4:** Run once and record the processed count; run again without adding files and confirm no new files are processed.

• **Step 5:** Handle the new cast_device field using the workbook's schema-evolution requirement, including Delta mergeSchema where appropriate.

• **Step 6:** Document checkpoint location, schema and evidence from both runs.

**Expected output:** Incremental Bronze playback table; checkpoint; evidence of no reprocessing on second run; cast_device support.

**Key concepts:** File streaming, availableNow, checkpoints, Delta append and schema evolution.

### *Q17. Real-time trending*

**Introduction:** Identify titles whose live audience is rising unusually quickly, rather than simply listing the most-watched titles.

• **Step 1:** Start python ott_stream_simulator.py --stream plays in a separate terminal.

• **Step 2:** Read the live start-event stream and enrich it with content metadata.

• **Step 3:** Count distinct viewers per title in 10-minute event-time windows sliding every minute; configure a watermark.

• **Step 4:** Build a batch baseline for the same hour over the previous seven days.

• **Step 5:** Compare current counts with the baseline and calculate growth ratio.

• **Step 6:** Apply the minimum 200-viewer condition, rank the top 20 and separate results by language.

• **Step 7:** Write the streaming output and document late-event handling.

**Expected output:** trending_now stream table with top 20 by language and growth ratio.

**Key concepts:** Event time, sliding windows, watermarks, approximate distinct counts and ranking.

### *Q20. Churn model and integrated pipeline*

**Introduction:** Predict subscribers likely to cancel and create a one-command pipeline for the project.

• **Step 1:** Define churn as no renewal within seven days after a subscription period ends.

• **Step 2:** Build features only from information available before the renewal date to avoid leakage.

• **Step 3:** Create features for four-week watch-minute trend, active days, titles started versus finished, failed searches, payment failures, plan, tenure and language.

• **Step 4:** Train an MLlib classifier with a time-based split; report AUC and important features.

• **Step 5:** Score subscribers renewing next week; rank by churn risk multiplied by plan value and output the top 50,000.

• **Step 6:** Integrate ingestion, Silver, quality gates, Gold and scoring in src/pipeline.py with logging and failure codes.

• **Step 7:** Coordinate input/output contracts with every module owner; take technical review from Rutik.

**Expected output:** Model report, weekly retention list, prioritized top 50,000 and one-command pipeline.

**Key concepts:** Churn labels, leakage prevention, feature engineering, time split, MLlib and pipeline integration.

# 5. Rutik — Playback Engineering and Optimization

Assigned: Q2, Q5, Q12, Q14

### *Q2. Clean playback events and correct device clocks*

**Introduction:** Produce reliable event timestamps despite duplicate events, incorrect device clocks and late offline uploads.

- **Step 1:** Read playback_events and inspect event types, timestamps, device IDs and download_id.
- **Step 2:** Remove duplicate records using event_id.
- **Step 3:** For online events only, calculate client_ts versus server_ts offset per device.
- **Step 4:** Use a robust statistic such as median offset to identify clock errors.
- **Step 5:** Create corrected event_ts using the device offset when reliable; fall back to server time when unreliable.
- **Step 6:** Preserve and label offline-download events separately.
- **Step 7:** Compare daily watch hours before and after correction.

**Expected output:** Silver playback_clean and a report on daily watch-hour changes.

**Key concepts:** Deduplication, median/percentile_approx, conditional logic and timestamp handling.

### *Q5. Viewing sessions and watch time*

**Introduction:** Calculate actual viewing time, excluding pauses, skipped scenes and implausibly long playback.

- **Step 1:** Use playback_clean and dim_content.
- **Step 2:** Order events by device and timestamp; use lag to calculate gaps between heartbeats.
- **Step 3:** Count heartbeat intervals only when the gap is up to six minutes while playing.
- **Step 4:** Treat forward seeks as skipped content.
- **Step 5:** Cap watch time per title at 1.5 times title duration and cut sessions after four hours without interaction.
- **Step 6:** Start a new session after 30 minutes of inactivity.
- **Step 7:** Calculate watched minutes per profile/content and completion ratio; validate pause, seek and inactivity cases.

**Expected output:** viewing_sessions and watch_time_daily with profile, content, minutes and completion.

**Key concepts:** Lag, window functions, sessionisation, interval sums and business-rule caps.

### *Q12. Detect account sharing*

**Introduction:** Identify accounts exceeding their plan's concurrent-screen allowance or showing suspicious simultaneous locations.

- **Step 1:** Join playback_clean, viewing_sessions and subscription_periods.
- **Step 2:** Find overlapping sessions and calculate maximum simultaneous devices per account.
- **Step 3:** Calculate distinct devices and cities per account per month.
- **Step 4:** Compare observed concurrency with max_screens in reference/plans.csv.
- **Step 5:** Flag accounts exceeding the limit on five or more days or streaming simultaneously from distant cities.
- **Step 6:** Exclude offline playback from concurrent-stream calculations.
- **Step 7:** Estimate possible additional revenue if flagged households converted.

**Expected output:** account_sharing_flags and revenue estimate.

**Key concepts:** Overlapping intervals, time slots, distinct counts and plan lookups.

### *Q14. Handle live-cricket data skew*

**Introduction:** Diagnose and reduce Spark slowdown caused by one content_id dominating playback and CDN data.

- **Step 1:** Count events by content_id for the specified cricket-final date and identify the hot key.
- **Step 2:** Inspect Spark UI and record skew in task duration or partition sizes.
- **Step 3:** Run the relevant join and record baseline runtime.

- **Step 4:** Test Adaptive Query Execution skew-join handling and measure the result.
- **Step 5:** Try salting the dominant key on the large side and replicating matching rows on the smaller side across salt buckets.
- **Step 6:** Compare runtime and correctness for baseline, AQE and salting.
- **Step 7:** Partition by event date/hour; test Z-ordering by content_id where supported.
- **Step 8:** Calculate CDN cost per title and region using bytes multiplied by rate per GB.

**Expected output:** Performance comparison report and cdn_cost_by_title.

**Key concepts:** Data skew, AQE, salting, partitioning, Z-ordering and cost allocation.

# 6. Harshvardhan — Content Modelling and Discovery

Assigned: Q1, Q8, Q10, Q16

### *Q1. Ingest the nested content catalogue*

**Introduction:** Flatten nested catalogue JSON into queryable title, episode, language, genre and cast datasets.

- **Step 1:** Read catalogue JSON using an explicit nested schema.
- **Step 2:** Create dim_content with one row per title or episode; carry series and season information to episode rows.
- **Step 3:** Explode languages into content_language.
- **Step 4:** Explode genres into content_genre.
- **Step 5:** Explode cast members and roles into content_cast.
- **Step 6:** Standardize genre names using reference/genre_mapping.csv.
- **Step 7:** Calculate title counts and total hours by language and genre; validate keys and duplicates.

**Expected output:** dim_content, content_genre, content_language and content_cast.

**Key concepts:** Nested schemas, explode, bridge tables and many-to-many modelling.

### *Q8. Track content tier and licence history*

**Introduction:** Preserve content attribute history so analysis uses the tier and partner valid at viewing time.

- **Step 1:** Compare monthly catalogue snapshots to identify tier, partner and title changes.
- **Step 2:** Build a Type 2 SCD with valid_from and valid_to.
- **Step 3:** Join watch time to the content version valid at the viewing timestamp.
- **Step 4:** Map viewing city to state using reference/city_state.csv.
- **Step 5:** Compare viewing state and timestamp with licence period to identify violations.
- **Step 6:** Use Delta time travel to show catalogue as it existed on 1 August.
- **Step 7:** Validate that each viewing event matches no more than one applicable version.

**Expected output:** dim_content_scd2 and licence-violation report.

**Key concepts:** SCD2, point-in-time joins, range checks and Delta time travel.

### *Q10. Build regional Top 10 rankings*

**Introduction:** Rank titles by genuine local audience, avoiding episode duplication and domination by heavy viewers.

- **Step 1:** Use watch_time_daily, dim_content and users.
- **Step 2:** Roll episode viewing up to its parent series.
- **Step 3:** Count distinct viewers with at least two minutes watched; use hours as tie-breaker.
- **Step 4:** Exclude kids profiles from the general list and create a separate kids list.
- **Step 5:** Require 50 viewers for a state list; otherwise fall back to national ranking.
- **Step 6:** Rank titles by state and date; compare with yesterday to calculate rank_change.

**Expected output:** top10_daily with date, state, rank, content, viewers and rank_change.

**Key concepts:** Rollups, dense_rank, fallback joins and day-over-day comparison.

### *Q16. Build collaborative-filtering recommendations*

**Introduction:** Recommend titles based on similar profiles' viewing behaviour rather than genre popularity alone.

- **Step 1:** Use watch_time_daily, ratings and dim_content.
- **Step 2:** Build implicit feedback per profile/title from completion and rewatching; include ratings where available.
- **Step 3:** Convert profile and content string IDs to integers using indexing.
- **Step 4:** Train Spark MLlib ALS with implicitPrefs=True.
- **Step 5:** Hold out each profile's last five titles and measure precision at 10.

- **Step 6:** Generate recommendations and remove already-watched titles.
- **Step 7:** Filter titles not licensed in the user's region and adult titles for kids profiles.
- **Step 8:** Compare against most-popular-in-language baseline.

**Expected output:** recommendations table and evaluation report.

**Key concepts:** Implicit feedback, StringIndexer, ALS, ranking evaluation and business-rule filtering.

# 7. Kshitij Jaisawal — Subscriptions, Finance and Live Ad Pacing

Assigned: Q6, Q11, Q15, Q18

### *Q6. Build the subscription lifecycle*

**Introduction:** Turn billing events into subscription periods and daily states while handling retries and duplicates.

- **Step 1:** Read subscription_events and inspect event types, plan, price, gateway and timestamps.
- **Step 2:** Deduplicate and order events per user.
- **Step 3:** Create subscription periods with user, plan, status, start_date and end_date.
- **Step 4:** Model trial, paid, paused, grace and churned states.
- **Step 5:** Treat payment failure followed by renewal within seven days as continuous subscription.
- **Step 6:** Calculate subscribers by plan and status for each date.
- **Step 7:** Reconcile paid subscriber counts with gateway settlements.

**Expected output:** subscription_periods and subscribers_daily.

**Key concepts:** Event-to-state modelling, ordering, grace periods and reconciliation.

### *Q11. Subscriber cohorts and monthly revenue*

**Introduction:** Measure retention by acquisition cohort and explain the sources of MRR movement.

- **Step 1:** Assign each subscriber to the month of their first paid event.
- **Step 2:** Build cohort retention by cohort month and months since joining.
- **Step 3:** Calculate the percentage still paying in each later month.
- **Step 4:** Calculate monthly recurring revenue.
- **Step 5:** Split MRR movement into new, expansion, contraction, churned and reactivated revenue.
- **Step 6:** Compare retention across acquisition channels.
- **Step 7:** Validate totals against subscription periods and daily subscriber states.

**Expected output:** cohort_retention and mrr_bridge_monthly.

**Key concepts:** Cohort analysis, window functions, months_between, pivot and MRR decomposition.

### *Q15. Calculate content partner royalties*

**Introduction:** Allocate subscription and advertising revenue to content partners using watch time and contract terms.

- **Step 1:** Use premium watch time, content history, subscription revenue, ad revenue and partner reference data.
- **Step 2:** Calculate each partner's share of premium viewing minutes per month.
- **Step 3:** Allocate subscription revenue pool using revenue-share percentage and viewing share.
- **Step 4:** Add eligible ad-revenue share from free-tier viewing.
- **Step 5:** Apply quarterly minimum guarantee: pay the higher of earned royalty and guarantee.
- **Step 6:** Handle decimal rounding and reconcile total paid, including top-ups.
- **Step 7:** Produce a per-title statement for the disputed partner and verify totals to the paisa.

**Expected output:** partner_royalties_quarterly and per-partner statements.

**Key concepts:** Proportional allocation, point-in-time partner mapping, guarantees and decimal arithmetic.

### *Q18. Live ad pacing and frequency capping*

**Introduction:** Monitor live impressions to prevent users exceeding caps and campaigns spending too quickly.

- **Step 1:** Start python ott_stream_simulator.py --stream ads.
- **Step 2:** Read live impressions and campaign reference data.
- **Step 3:** Maintain impression counts per user, campaign and date using streaming state.
- **Step 4:** Identify users who have reached the daily cap.

- **Step 5:** Calculate spend and compare with ideal pace across match hours.
- **Step 6:** Alert when campaign pace is above 120% or below 70% of ideal pace.
- **Step 7:** Reset state at midnight, for example by including date in the state key.
- **Step 8:** Validate impression counts, spend and reset behaviour.

**Expected output:** capped_users_live and pacing_alerts.

**Key concepts:** Stateful aggregation, frequency caps, spend calculations and date-based reset.

# 8. Siddhi — CDN Processing and Search Analytics

Assigned: Q3, Q7

### *Q3. Parse raw CDN access logs*

**Introduction:** Standardize two providers' differently formatted logs for bandwidth analysis.

• **Step 1:** Read raw CDN text files and inspect sample lines from both providers.

• **Step 2:** Write one regular expression per provider to extract timestamp, CDN, edge, client hash, content ID, quality, segment, status, bytes and user agent.

• **Step 3:** Parse both formats into one common schema.

• **Step 4:** Map edge names to regions using reference/cdn_edges.csv.

• **Step 5:** Quarantine unmatched lines and count malformed records per file.

• **Step 6:** Aggregate bytes by hour, CDN, region and content.

• **Step 7:** Validate parsed values against sample raw lines.

**Expected output:** Bronze cdn_requests and Silver cdn_usage_hourly.

**Key concepts:** spark.read.text, regexp_extract, regex, reference mapping, quarantine and aggregation.

### *Q7. Clean search logs and identify content gaps*

**Introduction:** Find genuine unmet content demand while distinguishing it from spelling variants and title typos.

• **Step 1:** Read search_logs and dim_content.

• **Step 2:** Normalize queries: lowercase, trim, remove punctuation and reduce repeated letters.

• **Step 3:** Group similar queries using soundex or Levenshtein distance, initially within the same first letter.

• **Step 4:** Calculate search-to-play conversion when a play follows within ten minutes.

• **Step 5:** Identify zero-result query groups and rank the top 50 per week.

• **Step 6:** Compare zero-result queries with existing title names using edit distance.

• **Step 7:** Document leading genuine content gaps and representative searches.

**Expected output:** search_clean, search_conversion and weekly content-gap list.

**Key concepts:** Text normalization, fuzzy matching, Levenshtein, time-window joins and ranking.

# 9. Ankita — Engagement Metrics, Advertising and Data Quality

Assigned: Q9, Q13, Q19

### *Q9. Calculate DAU, WAU and MAU*

**Introduction:** Create reliable active-user metrics that count a user once even when multiple devices are used.

• **Step 1:** Use watch_time_daily, users and subscribers_daily.

• **Step 2:** Define active as at least one minute watched in a day.

• **Step 3:** Calculate DAU using distinct user IDs per day.

• **Step 4:** Calculate rolling seven-day WAU and 30-day MAU.

• **Step 5:** Calculate stickiness as DAU divided by MAU.

• **Step 6:** Break down by plan, platform and state.

• **Step 7:** Compare exact and approximate distinct counts and quantify the difference.

• **Step 8:** Compare with reference/published_metrics.csv and investigate the month with the larger discrepancy.

**Expected output:** active_users_daily containing DAU, WAU, MAU and stickiness.

**Key concepts:** Distinct counts, rolling windows, approximate aggregation and multidimensional analysis.

### *Q13. Calculate advertising delivery and revenue*

**Introduction:** Measure campaign delivery, revenue, frequency violations and targeting compliance.

• **Step 1:** Read ad_impressions, ad_breaks, ad_campaigns, viewing_sessions and dim_content.

• **Step 2:** Calculate fill rate: breaks that showed an ad divided by total breaks served.

• **Step 3:** Calculate impressions, completion rate and revenue per campaign per day.

• **Step 4:** Count impressions per user/campaign/day and compare with the frequency cap.

• **Step 5:** Calculate violations and revenue to credit back.

• **Step 6:** Check whether impressions appeared on content matching target genres; ALL means unrestricted.

• **Step 7:** Compare delivery with planned daily budget and produce a pacing report.

**Expected output:** ad_delivery_daily, frequency_violations and campaign pacing report.

**Key concepts:** Multi-table joins, CPM revenue, frequency caps, genre membership and delivery metrics.

### *Q19. Data quality, kids-safety checks and tests*

**Introduction:** Find unsafe playback on kids profiles and validate key business metrics before publication.

• **Step 1:** Join playback_clean with profiles and content age ratings to find 16+ or 18+ plays on kids profiles.

• **Step 2:** Group violations by app version to identify patterns.

• **Step 3:** Add a quality gate: watch time per profile per day must not exceed 24 hours.

• **Step 4:** Check every play links to a known content ID.

• **Step 5:** Check royalty totals reconcile to the applicable pool.

• **Step 6:** Store published monthly metric values and compare recalculations.

• **Step 7:** Alert when a recalculation changes a published value by more than 0.5%.

• **Step 8:** Write pytest tests for watch-time calculation, subscription periods and royalty allocation; coordinate with Q5, Q6 and Q15 owners.

**Expected output:** Kids-safety report, quality results, published-metrics log and passing tests.

**Key concepts:** Compliance queries, quality gates, metric versioning, reconciliation and Pytest.

# 10. Shared integration checklist

| Area | Completion check |
|---|---|
| Environment | All members use compatible Python, Java, PySpark and Delta versions; shared SparkSession works. |
| Input contracts | Input files/tables, columns and data types are documented. |
| Output contracts | Output table names, schemas, paths and owners are agreed. |
| Lineage | Raw inputs remain unchanged; derived outputs can be traced to sources. |
| Restartability | Batch jobs can be rerun safely; streaming queries use stable individual checkpoints. |
| Validation | Row counts, nulls, duplicates, date ranges and business totals are checked. |
| Testing | Transformations have small test cases; required Q19 tests pass. |
| Integration | Pipeline runs in order and stops clearly if a quality gate fails. |
| Delivery | Each owner supplies code, output evidence, explanation and assumptions/limitations. |

## Final delivery requirements

- Working code for every assigned question in the shared repository.
- Expected tables, reports, model outputs or streaming outputs specified in the workbook.
- Validation evidence, such as representative output, counts, reconciliations and test results.
- A concise explanation of each implementation, its business rules and assumptions.
- An integrated pipeline and final demonstration coordinated by Ishan.

**Source basis:** StreamBharat – OTT Streaming Analytics Platform (Project 4) workbook. Question descriptions, inputs, outputs and concepts follow that workbook. Member allocation and coordination are the team's implementation plan.